4/11/2026

Conceptos Intervalo, operadores y Evaluación perezosa

Intervalo: un intervalo es un rango de valores entre dos límites. En Haskell no existe un tipo llamado "intervalo": se representa con listas generadas por rangos

Sintaxis

[inicio .. fin]

Ejemplo:

[1..5] -- [1,2,3,4,5]

Haskell llama a una función de la clase Enum llamada enumFromTo:

[1..5] == enumFromTo 1 5

Formas de escribir intervalos

[a..b]	Ejemplo: [1..5]
[a,b..c]	Ejemplo: [1,3..9]
[a..]	Ejemplo: [1..]
[a,b..]	Ejemplo [2,4..]

Un intervalo en Haskell es una lista generada por la clase Enum mediante la sintaxis [a..b]. No es un tipo especial: es una lista normal que puede ser infinita por que la evaluación perezosa solo produce los elementos que el consumidor demanda. El salto lo determina el segundo elemento en la forma [a,b..c], y para descender es obligatorio indicarlo.

Escaneado con CamScanner

Operador: Identificador formado exclusivamente por símbolos que representa una función y que puede usarse en notación infija.

Un operador es una función que se escribe entre sus dos argumentos.

1 + 2

Símbolos permitidos
! # $ % & * + . / < = > ? @ \ ^ | - ~ :

No debe contener: letras, números, paréntesis, corchetes ni llaves.

Precedencia: cada operador tiene un número del 0 al 9 más alta = se evalúa primero.

2 + 3 * 4 — 14 porque * (7) va antes que + (6)

Número	Operadores	Se agrupan
9	.	Derecha
8	^, **	Derecha
7	*, /, div, mod	Izquierda
6	+, -	Izquierda
5	:, ++	Derecha
4	==, <, >,	No se pueden agrupar
3	&&	Derecha
2	||	Derecha
0	$, $!	Derecha

Si no se agrupa nada: precedencia a, se agrupa por izquierda.

Escaneado con CamScanner

Asociatividad: cómo se agrupan los iguales. Solo importa cuando hay varios operadores del mismo número.

```
10 - 5 - 2      -- (10-5) - 2 = 3      (izquierda)
1 : 2 : [3]     -- 1 : (2 : [3]) = [1,2,3]  (derecha)
1 == 2 == 3     -- ERROR (No se puede)
```

¿Porque `1 == 2 == 3` es error? Porque `1 == 2` da True y `True == 3` no tiene sentido.

Forma correcta

```
1 == 2 && 2 == 3   -- False
```

Un operador en haskell es una función de dos argumentos escrita en medio. Su comportamiento depende de dos cosas: precedencia (número del 0 al 9, desde qué se evalúa primero) y asociatividad (desde cómo se agrupan los del mismo número). Las más importantes son $, ., :, ++, &&, || y los de comparación. Las secciones como (*2) fijan un argumento y dan una función de uno solo. Puedes crear operadores propios con `infixl`, `infixr` o `infix`.

Evaluación perezosa

Es una estrategia de evaluación en la que una expresión no se reduce hasta que su valor es demandado por el contexto. Haskell implementa esta estrategia mediante thunks (expresiones pendientes almacenadas en el heap) y call-by-need (evaluación con memorización del

Escaneado con CamScanner